Auto-Scaling and Self-Healing Cloud-Native Application using Kubernetes, Docker, Terraform and AWS

Student Details

- Student Name(s):
  - Myisha Siddiqui, 240905560
  - Manan Bhasin, 240905492
  - Rishabh Raj, 240905420
- Semester: 5th Semester
- Section: CSE - A
- Batch: *2028*
- Academic Year: 2026-27
- Department: School of Computer Engineering

Supervisor Details

- Supervisor Name: Dr. Krishnaraj Chadaga
- Department: School of Computer Engineering

Synopsis / Abstract

This project builds a small Spring Boot REST API, backed by a MySQL database, and shows how it can automatically handle changes in user traffic and unexpected failures without any human stepping in. The application is packaged using Docker and deployed on Kubernetes, a tool that manages multiple running copies of an application, on top of cloud infrastructure (AWS EC2 and RDS) provisioned automatically using Terraform, with GitHub Actions automating the build and deployment pipeline. When traffic increases, Kubernetes automatically starts more copies of the application to handle the load; when traffic goes down, it removes the extra copies to save resources. The project also shows that if one copy of the application suddenly stops working, Kubernetes detects this and starts a new copy in its place without any downtime for users. A live dashboard built using Prometheus and Grafana visually shows CPU usage, number of running copies, and traffic in real time. Finally, the project compares the estimated cloud cost of running the application with a fixed number of copies at all times versus running it with automatic scaling, to show the real-world, money-saving benefit of this approach.

Problem Statement

A single running copy of an application can only handle a limited number of users at a time. If traffic increases beyond this limit, users experience slow response times or the application may stop responding altogether. Additionally, if that single copy crashes due to an error, the entire application becomes unavailable until someone manually restarts it. Manually watching traffic and manually starting, stopping, or provisioning servers is slow, error-prone, and not practical in real-world systems.

This problem affects any modern web application or business that relies on cloud infrastructure to serve unpredictable traffic. Addressing it is important to avoid downtime, prevent a poor user experience during traffic spikes, and reduce unnecessary cloud spending, thereby improving both the reliability and the cost-efficiency of cloud-hosted applications.

Proposed Solution

The proposed solution is a cloud-hosted REST API application that automatically scales and heals itself under changing traffic and failure conditions. It consists of the following modules:

Application Module:
A Spring Boot REST API (for example, a student-records or task-management service) with GET and POST endpoints backed by a MySQL database. The application is kept stateless so that every running copy reads and writes to the same shared database, allowing any copy to safely handle any request.

Infrastructure Provisioning Module (Terraform + AWS):
Terraform scripts automatically provision the required AWS infrastructure - an EC2 instance to host the Kubernetes cluster and an RDS managed MySQL database - so the entire environment can be created or torn down with a single command, instead of manual setup.

Containerization Module (Docker):
The Spring Boot application is packaged into a Docker image containing the application code and everything it needs to run, ensuring it behaves identically in development, testing, and the final demo.

CI/CD Pipeline Module (GitHub Actions):
A GitHub Actions workflow automatically builds a new Docker image and redeploys it to

the Kubernetes cluster whenever code is pushed to the repository, removing the need for manual build-and-deploy steps.

Orchestration and Auto-Scaling Module (Kubernetes):
Kubernetes runs the application as multiple Pods (copies) on the provisioned EC2 instance. A Horizontal Pod Autoscaler rule automatically adds more Pods when CPU usage crosses a threshold and removes them when traffic drops, while continuous health checks let Kubernetes detect and replace a failed Pod automatically.

Monitoring Module (Prometheus and Grafana):
Prometheus continuously collects live metrics such as CPU usage, Pod count, and request rate, while Grafana turns this data into a live dashboard so that scaling and recovery events can be seen visually during the demo.

Cost Comparison Module:
Using publicly available AWS pricing, the estimated monthly cost of running the application with a fixed number of copies at all times is calculated and compared against the estimated cost with auto-scaling enabled, to show the real-world cost benefit of this approach.

Together, these modules demonstrate both the technical implementation of auto-scaling and fault tolerance on real cloud infrastructure, and their practical cost benefit, using tools that are widely used in the software industry today.

Tools and Technologies

- Cloud Service Provider: Amazon Web Services (AWS) - Free Tier
- Cloud Managed Services: Amazon EC2 (managed compute instance), Amazon RDS for MySQL (managed relational database), Terraform (Infrastructure as Code for provisioning EC2 and RDS), GitHub Actions (CI/CD pipeline for automated build and deployment)
- Databases: Amazon RDS for MySQL (managed database service)
- Source Version Control: Git and GitHub
- Tools: Docker (containerization), Kubernetes (container orchestration and auto-scaling), Prometheus (metrics collection), Grafana (monitoring dashboard), k6 (load testing)

Expected Outcomes

The expected outcome is a fully working, cloud-deployed REST API application that scales and heals itself automatically. The project will be considered successful if: (1) AWS infrastructure is provisioned automatically using Terraform, (2) code changes are automatically built and deployed using a GitHub Actions pipeline, (3) Kubernetes automatically adds and removes Pods in response to a simulated traffic spike, (4) a deliberately stopped Pod is automatically detected and replaced with zero downtime for users, (5) a live Grafana dashboard clearly shows these scaling and recovery events as they happen, and (6) a clear cost comparison shows the savings of auto-scaling over a fixed-capacity deployment.

Tentative Timeline

Week #	Phase/Tasks
1	Finalize application design, REST API endpoints, and database schema
2	Build and test the Spring Boot REST API locally with a MySQL database
3	Write Terraform scripts to provision the AWS EC2 instance and RDS MySQL database
4	Containerize the application with Docker; set up GitHub Actions CI/CD pipeline to build and push the Docker image
5	Set up Kubernetes on the provisioned EC2 instance and deploy the application as Pods
6	Configure the Horizontal Pod Autoscaler; set up Prometheus and Grafana monitoring dashboard
7	Perform load testing and fault-tolerance testing; record results
8	Calculate the cost comparison, finalize documentation, and prepare the final presentation/demo